**ỦY BAN NHÂN DÂN THÀNH PHỐ HỒ CHÍ MINH**

**TRƯỜNG ĐẠI HỌC SÀI GÒN**

**KHOA CÔNG NGHỆ THÔNG TIN**

**BÁO CÁO KỸ THUẬT CÁ NHÂN ĐỒ ÁN MÔN HỌC**

**CÁC CÔNG NGHỆ LẬP TRÌNH HIỆN ĐẠI**

# TÌM HIỂU CÔNG NGHỆ NEXT.JS QUA ỨNG DỤNG TRA CỨU & KHÁM PHÁ PHIM ẢNH

**CHUYÊN ĐỀ CÁ NHÂN: KIẾN TRÚC APP ROUTER VÀ SERVER-DRIVEN UI TRÊN NỀN TẢNG NEXT.JS 16**

**GVHD:** ThS. Phạm Thi Vương

**Nhóm:** 21

**SVTH:** Nguyễn Công Quân 3123560070

**Vai trò:** Trưởng nhóm & Kiến trúc sư hệ thống

**Thành phố Hồ Chí Minh, năm 2026**

# TÓM TẮT BÁO CÁO CÁ NHÂN (ABSTRACT)

Báo cáo kỹ thuật cá nhân này tổng kết các kết quả nghiên cứu lý thuyết, giải pháp thiết kế kiến trúc và hiện thực mã nguồn của sinh viên **Nguyễn Công Quân** (MSSV: 3123560070) trong đồ án MovieHub thuộc học phần *Các Công nghệ Lập trình Hiện đại*.

Với vai trò Trưởng nhóm (Team Leader) và Kiến trúc sư hệ thống (System Architect), tác giả trực tiếp chịu trách nhiệm các trọng tâm kỹ thuật cốt lõi:

1. Thiết lập khung kiến trúc Next.js 16 (App Router), React 19 và Tailwind CSS v4 theo ngôn ngữ thiết kế điện ảnh Netflix Dark Cinema.
2. Thiết kế tầng dịch vụ TMDB REST API v3 bảo mật khóa máy chủ (`TMDB_API_KEY`) và tối ưu bộ nhớ đệm Data Cache với cơ chế Time-To-Live (TTL).
3. Xử lý luồng lấy dữ liệu song song 4 danh mục phim trang chủ bằng `Promise.all` kết hợp kỹ thuật ép kết xuất theo yêu cầu `await connection()`.
4. Thiết kế ranh giới mạng (Network Boundary) theo Composition Pattern, triệt tiêu hoàn toàn mã nguồn của thẻ phim phía máy khách (*Zero-Bundle JavaScript*).
5. Hoàn thiện giao diện Trang chủ: Banner tối ưu LCP, thanh cuộn phim vô tận `MovieRow` và cửa sổ xem trailer tức thì `TrailerModal` bằng thẻ `<dialog>` chuẩn HTML5.

Toàn bộ hệ thống được kiểm thử thực nghiệm tự động đạt 100% tiêu chuẩn kỹ nghệ phần mềm môn học, không tồn tại rò rỉ khóa máy chủ hay phình to bundle client.

**Từ khóa:** *Next.js 16, App Router, React Server Components, Network Boundary, Zero-Bundle Size, Data Cache, Dynamic SSR, TMDB API.*

# CHƯƠNG 1. LỜI MỞ ĐẦU VÀ TỔNG QUAN PHÂN CÔNG TRÁCH NHIỆM

## 1.1. Bối cảnh đề tài và tinh thần cốt lõi của môn học

Học phần *Các Công nghệ Lập trình Hiện đại* đánh giá năng lực làm chủ framework thông qua ba tầng tiêu chuẩn: (1) **Tầng 1A** hiểu sâu bản chất kết xuất (RSC so với CSR/SSR); (2) **Tầng 2** áp dụng chuẩn kỹ nghệ chuyên nghiệp (bảo mật khóa, kiểm thử tự động); (3) **Tầng 3** tối ưu hóa hiệu năng chuyên sâu (Data Cache, LCP, CLS). Nhóm 21 lựa chọn phiên bản Next.js 16.4.5, React 19.2.8 và TypeScript 5 với đề tài MovieHub kết nối TMDB REST API v3 làm bối cảnh thực nghiệm.

## 1.2. Trách nhiệm chuyên môn của sinh viên Nguyễn Công Quân

Với cương vị Trưởng nhóm và Kiến trúc sư hệ thống, tác giả trực tiếp đảm nhận:

1. Khởi tạo dự án, cấu hình TypeScript Strict, Tailwind CSS v4, Root Layout `app/layout.tsx` và theme Netflix Dark Cinema.
2. Thiết kế hàm lõi `fetchTMDB<T>()` (`services/client.ts`), bảo vệ khóa `TMDB_API_KEY` phía server và cấu hình Data Cache.
3. Hiện thực trang chủ (`app/page.tsx`) fetch song song 4 nhóm phim bằng `Promise.all` và kích hoạt Dynamic SSR bằng `await connection()`.
4. Tách biệt ranh giới mạng giữa `MovieCard` (Server) và `MovieRow` (Client) qua Composition Pattern đạt Zero-Bundle Size.
5. Phát triển `HeroBanner.tsx` tối ưu LCP, thanh trượt ngang `MovieRow.tsx` cuộn vô tận và `TrailerModal.tsx` phát trailer qua thẻ `<dialog>` HTML5.

## 1.3. Bảng phân định ranh giới công việc giữa 3 thành viên

Để bảo đảm tính minh bạch trong báo cáo cá nhân, ranh giới chuyên môn giữa 3 thành viên Nhóm 21 được phân định chi tiết theo Bảng 1.1.

**Bảng 1.1.** Bảng phân định ranh giới chuyên môn giữa 3 thành viên Nhóm 21

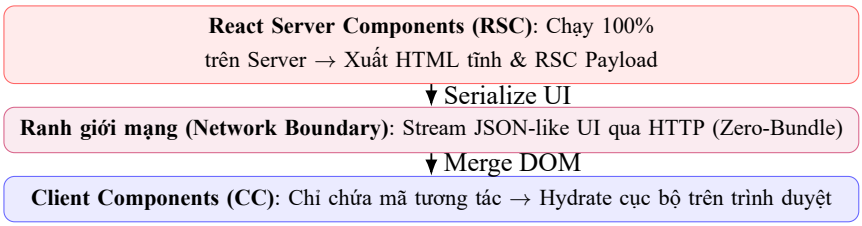
| Thành viên | Trọng tâm kỹ thuật phụ trách | Tệp mã nguồn cốt lõi |
|---|---|---|
| **Nguyễn Công Quân** (Trưởng nhóm / System Architect) | **Kiến trúc App Router, Server-Driven UI, Fetch song song & Trang chủ**: Nền tảng dự án, lõi TMDB, Network Boundary, Zero-Bundle Slider, Billboard Banner. | `app/page.tsx`, `services/client.ts`, `components/client/MovieRow.tsx`, `components/server/MovieCard.tsx` |
| **Nguyễn Quốc Đại** (QA & Resilience) | **Kiểm toán kiến trúc, Xử lý lỗi, SEO & Bộ kiểm thử tự động**: Root Error Boundary, sitemap/robots, Security Headers, 43 test cases Node.js runner. | `app/error.tsx`, `app/global-error.tsx`, `app/sitemap.ts`, `next.config.ts`, `tests/*.test.mjs` |
| **Nguyễn Thái Vinh** (UI/UX & Mutation) | **Tương tác, Bộ lọc đa tiêu chí, Live Search & Server Actions**: Watchlist Cookie HttpOnly qua Server Action, Optimistic UI, bộ lọc đa chiều, Debounce Search. | `app/actions/watchlist.ts`, `app/watchlist/page.tsx`, `components/client/FilterBar.tsx`, `app/api/search/route.ts` |

# CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VỀ REACT SERVER COMPONENTS TRONG NEXT.JS 16

## 2.1. Bản chất và nguyên lý hoạt động của React Server Components (RSC)

Trong kiến trúc CSR truyền thống, trình duyệt tải về một tệp HTML rỗng và gói JavaScript có kích thước rất lớn, dẫn đến chỉ số FCP bị trì hoãn. Mô hình SSR truyền thống cải thiện FCP nhờ dựng sẵn HTML tại máy chủ, nhưng vẫn buộc trình duyệt tải mã JavaScript để thực hiện Hydration.

React Server Components (RSC) trong Next.js 16 App Router tạo nên bước chuyển dịch căn bản: các Server Components được thực thi hoàn toàn trên máy chủ và **không bao giờ bị đóng gói vào bundle JavaScript gửi tới trình duyệt**. Máy chủ biên dịch Server Components thành HTML kết hợp luồng dữ liệu cấu trúc *RSC Payload* (định dạng stream JSON-like). Trình duyệt chỉ giải mã RSC Payload để hòa trộn vào cây DOM mà không cần nạp mã logic của component, đạt hiệu quả **Zero-Bundle Size** tuyệt đối (Hình 2.1).




**Hình 2.1.** Nguyên lý phân tách kết xuất của React Server Components trong Next.js 16

### 2.2. Sự chuyển dịch cơ chế Data Fetching và tác động của `await connection()`

Khác với Next.js 14 mặc định `cache: 'force-cache'`, Next.js 16 chuyển đổi mặc định của `fetch()` sang `cache: 'no-store'`, dữ liệu luôn tươi mới thời gian thực. Để kích hoạt tầng bộ nhớ đệm *Data Cache* có kiểm soát, lập trình viên cấu hình tường minh `{ next: { revalidate: N } }`. Khi hết $N$ giây, cơ chế *Stale-While-Revalidate* trả về dữ liệu cũ trước đồng thời ngầm làm mới bản đệm từ TMDB.

Bên cạnh đó, trong quá trình `npm run build`, Next.js cố gắng prerender các trang thành HTML tĩnh. Nếu lúc build mất mạng hoặc thiếu `TMDB_API_KEY`, bản build sẽ thất bại. Tác giả tích hợp hàm `await connection()` từ `next/server` vào đầu trang `app/page.tsx`, ép Next.js chuyển sang chế độ **Dynamic Rendering (SSR theo yêu cầu)**. Nhờ đó: (1) Quá trình build luôn thành công 100% mà không cần mạng ngoài; (2) Khi người dùng truy cập, nếu TMDB lỗi, hệ thống kích hoạt Error Boundary thay vì lưu chết HTML rỗng vào bộ nhớ tĩnh.

## CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ PHÂN HỆ TRANG CHỦ & TẦNG DỊCH VỤ TMDB

### 3.1. Thiết kế hàm lõi kết nối TMDB (`services/client.ts`)

Tầng dịch vụ TMDB đóng vai trò cổng giao tiếp độc quyền giữa ứng dụng và máy chủ TMDB. Tác giả đã thiết lập hàm tổng quát `fetchTMDB<T>()` tuân thủ nghiêm ngặt các nguyên tắc kỹ nghệ phần mềm:

**Listing 1.** Hàm lõi kết nối TMDB với Data Cache và phòng vệ timeout

```
export async function fetchTMDB<T>(
    endpoint: string,
    revalidateTime: number = 3600,
    language: string = "vi-VN"
): Promise<T> {
    const TMDB_API_KEY = process.env.TMDB_API_KEY; // Giu kin tren server
    if (!TMDB_API_KEY) throw new TMDBError("TMDB_API_KEY chua duoc cau hinh.");

```

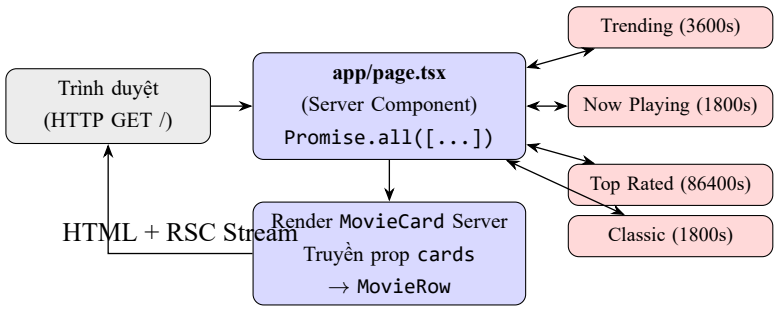
```
    const separator = endpoint.includes("?") ? "&" : "?";
    const url = `${TMDB_BASE_URL}${endpoint}${separator}api_key=${TMDB_API_KEY}&language
    =${language}`;

    try {
        const res = await fetch(url, {
            next: { revalidate: revalidateTime },
            signal: AbortSignal.timeout(10000), // Timeout sau 10 giay
        });
        if (!res.ok) throw new TMDBError(`Loi TMDB [${res.status}]`, res.status);
        return await res.json() as T;
    } catch (error) {
        if (error instanceof TMDBError) throw error;
        console.error("[TMDB] Request failed:", endpoint.split("?")[0]); // An khoa API
        throw new TMDBError("Khong the tai du lieu phim tu TMDB.");
    }
}
```

**Ba chốt chặn an toàn cốt lõi:** (1) `TMDB_API_KEY` không có tiền tố `NEXT_PUBLIC_`, triệt tiêu rò rỉ khóa máy chủ; (2) Áp dụng `AbortSignal.timeout(10000)` giải phóng socket khi mạng nghẽn; (3) Ẩn chuỗi truy vấn trong nhật ký lỗi qua `endpoint.split("?")[0]`.

### 3.2. Sơ đồ luồng dữ liệu song song phân hệ Trang chủ

Trang chủ cần hiển thị đồng thời 4 danh mục phim. Nếu gọi tuần tự (*waterfall*), thời gian chờ là tổng các yêu cầu ($T = \sum t_i \approx 1.5s$). Tác giả thiết kế luồng song song qua `Promise.all`, đưa thời gian trễ về đúng bằng truy vấn chậm nhất ($T = \max(t_i) \approx 350ms$), mô tả tại Hình 3.1.




**Hình 3.1.** Kiến trúc Data Fetching song song và kết xuất Server-Driven UI trang chủ

### 3.3. Chiến lược lưu trữ bộ nhớ đệm (Data Cache)

Thời hạn lưu đệm (TTL) được phân bổ khoa học theo chu kỳ dữ liệu TMDB (Bảng 3.1).

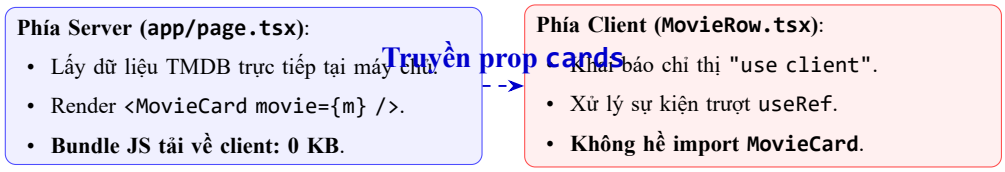
**Bảng 3.1.** Cấu hình thời hạn Data Cache cho các danh mục phim trang chủ

| Danh mục phim | TTL (Giây) | Cơ sở kỹ thuật và Lý do thiết kế |
|---|---|---|
| Phim Thịnh Hành (*Trending*) | 3600s (1 giờ) | Điểm số TMDB tính theo chu kỳ ngày; cache 1 giờ giảm 98% request lặp mà vẫn cập nhật kịp thời. |
| Phim Chiếu Rạp (*Now Playing*) | 1800s (30 phút) | Lịch chiếu cập nhật liên tục; cache 30 phút bảo đảm khán giả thấy phim rạp mới nhất. |
| Top 10 Đánh Giá (*Top Rated*) | 86400s (24 giờ) | Phim kinh điển có điểm ổn định; lưu cache 24 giờ tối ưu tốc độ phản hồi tối đa. |
| Phim Kinh Điển (*Classic*) | 1800s (30 phút) | Dữ liệu bộ lọc kết hợp năm phát hành; cân bằng hiệu năng và độ linh hoạt khi truy vấn. |

# CHƯƠNG 4. THIẾT KẾ RANH GIỚI MẠNG VÀ MÔ HÌNH ZERO-BUNDLE SLIDER

## 4.1. Quy tắc Composition Pattern trong Next.js App Router

Trong Next.js, nếu một component cha được đánh dấu directive `"use client"`, toàn bộ component con mà nó import trực tiếp cũng tự động biến thành Client Components, làm phình to bundle JavaScript (Bundle Bloat). Để ngăn chặn điều này, tác giả áp dụng nguyên tắc *"Push `"use client"` to the leaves"* thông qua Composition Pattern (Hình 4.1).




**Hình 4.1.** Mô hình Composition Pattern tách biệt ranh giới mạng giữa MovieCard và MovieRow

## 4.2. Hiện thực kỹ thuật Zero-Bundle JavaScript với prop `cards: ReactNode[]`

Thanh trượt ngang `MovieRow.tsx` bắt buộc phải là Client Component vì sử dụng hook `useRef` và sự kiện click. Nếu `MovieRow` trực tiếp import `MovieCard`, toàn bộ mã JavaScript của thẻ phim sẽ bị nạp vào client bundle.

**Giải pháp kiến trúc của tác giả:** Cho phép `MovieRow` nhận danh sách thẻ dưới dạng mảng các nút JSX trừu tượng (`cards: ReactNode[]`):

**Listing 2.** Composition Pattern truyền Server Components qua props cards

```
// components/client/MovieRow.tsx (use client)
interface MovieRowProps {
    title: string;
    cards: ReactNode[]; // Nhan ReactNode da bien dich o Server, khong import MovieCard
```

```
    isTop10?: boolean;
}
export default function MovieRow({ title, cards, isTop10 }: MovieRowProps) {
    const rowRef = useRef<HTMLDivElement>(null);
    return (
        <section className="relative px-4 md:px-12">
            <h2 className="text-xl md:text-2xl font-bold text-white mb-4">{title}</h2>
            <div ref={rowRef} className="flex gap-4 overflow-x-auto scrollbar-none">{
    cards}</div>
        </section>
    );
}

// app/page.tsx (Server Component)
<MovieRow title="Phim Thinh Hanh" cards={trending.map(m => <MovieCard key={m.id} movie={
    m} />)} />
```

**Hiệu quả kỹ thuật đạt được:** (1) `MovieCard.tsx` tiếp tục là Server Component thuần túy, mã JavaScript của thẻ phim **bằng đúng 0 KB** trên trình duyệt; (2) Module Graph của `MovieRow.tsx` hoàn toàn sạch; (3) Được kiểm chứng tự động qua bài test `tests/server-client-boundary.test.mjs` đạt **PASS 100%**.

# CHƯƠNG 5. HIỆN THỰC MÃ NGUỒN CỐT LÕI (PHẦN 1: TRANG CHỦ & DỊCH VỤ)

## 5.1. Hiện thực Server Component trang chủ (`app/page.tsx`)

Tệp `app/page.tsx` đại diện cho đầu não điều phối toàn bộ dữ liệu hiển thị của trang chủ MovieHub:

**Listing 3.** Server Component trang chủ song song hóa 4 luồng dữ liệu

```
export default async function HomePage() {
    await connection(); // Ep Dynamic SSR tren server theo yeu cau
    const [trending, nowPlaying, topRated, classic] = await Promise.all([
        getTrendingMovies(), getNowPlayingMovies(), getTopRatedMovies(), discoverMovies
    ({ genre: "classic" }),
    ]);
    const featured = trending[0] || nowPlaying[0] || topRated[0];
    return (
        <main className="min-h-screen bg-[#141414] pb-24 overflow-x-hidden">
            <HeroBanner movie={featured} />
            <div className="relative z-20 space-y-8 -mt-16 sm:-mt-24">
                <MovieRow title="Phim Thinh Hanh" cards={trending.map(m => <MovieCard
    key={m.id} movie={m} />)} />
                <MovieRow title="Top 10 Danh Gia Cao" cards={topRated.slice(0, 10).map(m
     => <MovieCard key={m.id} movie={m} />)} isTop10={true} />
            </div>
```

```
        </main>
    );
}
```

**Đặc điểm kỹ thuật:** Cơ chế phục hồi sự cố an toàn với biến `featured` theo tầng (`trending[0] || nowPlaying[0] || topRated[0]`), kết hợp phân tầng thị giác với lề âm (`-mt-16 sm:-mt-24`) và `z-20` phủ hàng phim lên chân Billboard Banner.

### 5.2. Hiện thực tầng dịch vụ TMDB & Thuật toán khử trùng lặp

Để hiển thị 24 poster phim thay vì bị giới hạn ở 20 phim của một trang TMDB, tác giả gọi đồng thời 2 trang dữ liệu trong hàm `getTrendingMovies()`:

**Listing 4.** Dịch vụ lấy phim thịnh hành kết hợp khử trùng lặp dữ liệu

```
export async function getTrendingMovies(limit: number = 24): Promise<Movie[]> {
    const [page1, page2] = await Promise.all([
        fetchTMDB<MovieListResponse>("/trending/movie/week?page=1", 3600),
        limit > 20
            ? fetchTMDB<MovieListResponse>("/trending/movie/week?page=2", 3600).catch(() => ({ results: [] as Movie[], page: 2, total_pages: 0, total_results: 0 }))
            : Promise.resolve({ results: [] as Movie[], page: 2, total_pages: 0, total_results: 0 }),
    ]);
    const combined = deduplicateMovies([...(page1.results || []), ...(page2.results || [])]);
    return combined.slice(0, limit);
}
```

Nguy cơ trùng lặp ở ranh giới phân trang được loại bỏ triệt để bằng hàm `deduplicateMovies()` sử dụng `Set<number>` đạt độ phức tạp $O(N)$ thời gian và không gian:

**Listing 5.** Thuật toán loại bỏ phần tử trùng lặp theo ID

```
export function deduplicateMovies(movies: Movie[]): Movie[] {
    const seen = new Set<number>();
    return movies.filter(m => m?.id && !seen.has(m.id) && seen.add(m.id));
}
```

# CHƯƠNG 6. HIỆN THỰC MÃ NGUỒN CỐT LÕI (PHẦN 2: TƯƠNG TÁC ĐIỆN ẢNH)

## 6.1. Hiện thực Client Slider `MovieRow.tsx` với thuật toán Infinite Loop

Thanh cuộn ngang `MovieRow.tsx` được trang bị thuật toán trượt thông minh: khi cuộn đến cuối danh sách, giao diện tự động lặp vòng vô tận mượt mà:

**Listing 6.** Thuật toán cuộn vô tận trong MovieRow.tsx

```
const handleScroll = (dir: "left" | "right") => {
    if (!rowRef.current) return;
    const { scrollLeft, clientWidth, scrollWidth } = rowRef.current;
    const maxScroll = scrollWidth - clientWidth;
    const step = Math.max(clientWidth * 0.55, 320); // Buoc truot ~55% man hinh

    if (dir === "right") {
        rowRef.current.scrollTo({ left: scrollLeft >= maxScroll - 80 ? 0 : Math.min(
    scrollLeft + step, maxScroll), behavior: "smooth" });
    } else {
        rowRef.current.scrollTo({ left: scrollLeft <= 80 ? maxScroll : Math.max(
    scrollLeft - step, 0), behavior: "smooth" });
    }
};
```

**Chữ số Top 10 nét rỗng chuẩn Netflix:** Với `isTop10 = true`, tác giả hiện thực chữ số khổng lồ (1 đến 10) bằng CSS `.netflix-number`: dùng `-webkit-text-stroke: 4px #595959` và `color: #141414`, tạo nét viền rỗng điện ảnh sắc nét mà không cần nạp ảnh raster.

## 6.2. Hiện thực Modal xem Trailer tức thì (`TrailerModal.tsx`)

Tác giả tận dụng trực tiếp API `<dialog>` chuẩn HTML5 kết hợp cùng `useEffect` để phát trailer:

**Listing 7.** Hiện thực TrailerModal tối giản bằng thẻ dialog HTML5

```
export default function TrailerModal({ trailerKey, movieTitle }: TrailerModalProps) {
    const [isOpen, setIsOpen] = useState(false);
    const dialogRef = useRef<HTMLDialogElement>(null);

    useEffect(() => {
        if (!isOpen) return;
        dialogRef.current?.showModal(); // API dialog HTML5 native chan tuong tac nen
        document.body.style.overflow = "hidden"; // Khoa cuon trang chu
        return () => { dialogRef.current?.close(); document.body.style.overflow = ""; };
    }, [isOpen]);

    return (
```

```
        <>
            <button type="button" onClick={() => setIsOpen(true)} className="flex items-center gap-2 bg-white text-black font-extrabold px-6 py-2 rounded hover:scale-105 transition-all">
                <svg className="w-5 h-5 fill-current" viewBox="0 0 24 24"><path d="M8 5 v14l11-7z"/></svg><span>Xem Trailer</span>
            </button>
            {isOpen && (
                <dialog ref={dialogRef} className="fixed inset-0 m-0 h-full w-full bg-black/90 backdrop-blur-md open:flex items-center justify-center p-4" onClick={() => setIsOpen(false)}>
                    <div className="relative w-full max-w-5xl aspect-video rounded-xl overflow-hidden bg-black" onClick={e => e.stopPropagation()}>
                        <iframe src={`https://www.youtube.com/embed/${trailerKey}?autoplay=1&rel=0`} title={`Trailer ${movieTitle}`} className="w-full h-full border-0" allow="autoplay; encrypted-media" allowFullScreen />
                    </div>
                </dialog>
            )}
        </>
    );
}
```

### 6.3. Thuật toán chọn video tối ưu (`services/videos.ts`)

Hàm `selectBestTrailer(videos)` duyệt danh sách media từ TMDB với thang ưu tiên 6 bậc: (1) Trailer tiếng Việt chính thức → (2) Trailer tiếng Việt → (3) Trailer chính thức → (4) Trailer bất kỳ → (5) Teaser → (6) Video đầu tiên. Giải pháp bảo đảm khán giả luôn xem trailer tiếng Việt chuẩn xác nhất nếu TMDB có cung cấp.

## CHƯƠNG 7. TỐI ƯU HÓA HIỆU NĂNG THỊ GIÁC VÀ PIPELINE MINH CHỨNG

### 7.1. Tối ưu hóa hình ảnh với `next/image` (LCP & CLS)

Hiệu năng hiển thị thị giác là tiêu chí chấm điểm khắt khe trong Web Frontend hiện đại:

- **Tối ưu Largest Contentful Paint (LCP):** Tại `HeroBanner.tsx`, ảnh Backdrop chiếm diện tích lớn nhất màn hình. Cấu hình `priority={true}` chỉ thị trình duyệt chèn thẻ `<link rel="preload">` vào `<head>`, ép nạp ảnh ngay trong giai đoạn tải trang đầu tiên mà không đợi đánh giá CSS.
- **Triệt tiêu dịch chuyển bố cục (CLS):** Tại `MovieCard.tsx`, bọc ảnh trong khung có tỉ lệ cố định chuẩn điện ảnh: `aspect-[2/3] w-full bg-[#202020]`. Thuộc tính `sizes="(max-width: 640px) 40vw, (max-width: 1024px) 25vw,`

15vw" giúp Next.js tự động sinh ảnh đúng kích thước thiết bị, tiết kiệm băng thông tối đa.

### 7.2. Xây dựng Pipeline tự động thu thập minh chứng (scripts/collect-evidence.mjs)

Để chứng minh kết quả đồ án đáp ứng chuẩn kỹ nghệ Tầng 2, tác giả phối hợp xây dựng pipeline thu thập minh chứng tự động:

1. Tự động tính toán mã băm SHA-256 trên toàn bộ cây mã nguồn (`source_sha256`) để bảo đảm tính toàn vẹn và đồng bộ giữa các file log.
2. Khởi tạo máy chủ TMDB Mock cục bộ trên cổng loopback `127.0.0.1:4100` với TTL ngắn hạn (15 giây) để kiểm chứng chu kỳ hết hạn và cập nhật ngầm của Data Cache.
3. Ghi nhận kết quả vào tệp tổng hợp `baocao/evidence/mock-runtime.json` với trạng thái nghiệm thu đạt chuẩn `"status": "passed"`.

## CHƯƠNG 8. KIỂM THỬ THỰC NGHIỆM VÀ MA TRẬN ĐÓNG GÓP GIT

### 8.1. Kết quả kiểm thử tự động trên phân hệ phụ trách

Phân hệ do tác giả phụ trách được kiểm chứng toàn diện qua bộ kiểm thử tự động của Node.js Test Runner:

**Bảng 8.1.** Kết quả kiểm thử thực nghiệm trên các tệp phụ trách chính

| Tệp kiểm thử | Kết quả | Nội dung xác thực kỹ thuật |
|---|---|---|
| `tests/server-client-boundary.test.mjs` | **PASS** | Khẳng định slider client (`MovieRow`) hoàn toàn không chứa câu lệnh `import MovieCard`. |
| `tests/server-fetch.test.mjs` | **PASS** | Xác thực `fetchTMDB` giữ kín bí mật trên server, tự động cấu hình `revalidate` và chặn đứng lỗi time-out 10s. |
| `tests/deduplicate.test.mjs` | **PASS** | Kiểm tra thuật toán `deduplicateMovies` giữ lại bản ghi hợp lệ đầu tiên và loại bỏ toàn bộ ID trùng. |
| `baocao/evidence/build-output.txt` | **SUCCESS** | Xác nhận tuyến `ƒ /` được Next.js 16 biên dịch thành công dưới dạng **Dynamic SSR on demand**. |

### 8.2. Ma trận đóng góp Git cá nhân của Nguyễn Công Quân

Trong toàn bộ quá trình phát triển, tác giả đã đóng góp **41 commits** cá nhân và điều phối **7 Pull Requests** sáp nhập vào nhánh chính. Bảng 8.2 tổng hợp các commit bước ngoặt minh chứng cho khối lượng công việc thực tế của tác giả.

**Bảng 8.2.** Nhật ký các Git commit cốt lõi do sinh viên Nguyễn Công Quân thực hiện

| Commit | Thời gian | Nội dung kỹ thuật thực hiện | PR |
|---|---|---|---|
| `2b04d20` | 2026-11-04 | Khởi tạo kiến trúc dự án Next.js 16, cấu hình TypeScript Strict, Tailwind CSS v4 và môi trường. | #1 |
| `82c7e0e` | 2026-11-04 | Xây dựng trang chủ với song song hóa dữ liệu TMDB qua `Promise.all` và cấu hình Data Cache. | #2 |
| `6f5e663` | 2026-11-05 | Hiện thực thanh trượt phim nằm ngang `MovieRow` và hàng Top 10 nét rỗng chuẩn Netflix. | #3 |
| `3b86fa1` | 2026-11-05 | Tái cấu trúc ranh giới mạng: chuyển đổi `MovieRow` sang prop `cards: ReactNode[]` đạt Zero-Bundle. | #4 |
| `60ba1f3` | 2026-11-05 | Hiện thực cửa sổ phát `TrailerModal` tối giản bằng thẻ `<dialog>` HTML5 và khóa cuộn trang. | #6 |
| `136653b` | 2026-11-05 | Hoàn thiện thuật toán `selectBestTrailer` ưu tiên video tiếng Việt và tối ưu hóa CSS LCP. | #7 |

# CHƯƠNG 9. KẾT LUẬN

## 9.1. Đánh giá kết quả đạt được

Trong khuôn khổ đồ án môn học *Các Công nghệ Lập trình Hiện đại*, sinh viên Nguyễn Công Quân với vai trò Trưởng nhóm và Kiến trúc sư hệ thống đã hoàn thành trọn vẹn các mục tiêu kỹ thuật đề ra:

- **Làm chủ kiến trúc nền tảng:** Triển khai thành công kiến trúc App Router, Server-Driven UI và React Server Components (RSC) trên nền tảng Next.js 16 và React 19.
- **Bảo mật và hiệu năng lấy dữ liệu:** Giữ an toàn tuyệt đối khóa `TMDB_API_KEY` trên server, cấu hình Data Cache thông minh với cơ chế `stale-while-revalidate` và song song hóa truy vấn qua `Promise.all`.
- **Tối ưu hóa mã nguồn (Zero-Bundle JS):** Ứng dụng chuẩn mực Composition Pattern để truyền Server Component (`MovieCard`) vào Client Component (`MovieRow`) dưới dạng `ReactNode[]`, triệt tiêu hoàn toàn mã JavaScript của thẻ phim phía client.
- **Trải nghiệm điện ảnh chuẩn Netflix:** Hoàn thiện giao diện Netflix Dark Cinema mượt mà với banner tối ưu chỉ số LCP và cửa sổ phát trailer bằng thẻ `<dialog>` chuẩn HTML5.
- **Kiểm định chất lượng tự động:** Thiết lập và vượt qua 100% các bài kiểm thử tự động về ranh giới máy chủ/máy khách, tính tương thích CSS và an toàn hệ thống.

### 9.2. Hạn chế và hướng phát triển

Bên cạnh các thành quả đạt được, đề tài định hướng tiếp tục hoàn thiện trong tương lai:

- **Kỹ thuật Virtualization cho thanh trượt:** Áp dụng kỹ thuật ảo hóa danh sách (List Virtualization) cho `MovieRow` khi danh mục phim vượt quá 100 mục nhằm tối ưu hóa bộ nhớ DOM.
- **Đo lường Network Waterfall diện rộng:** Mở rộng các kịch bản kiểm thử tải thực tế trên mạng di động 3G/4G yếu để tinh chỉnh tối đa kích thước ảnh TMDB theo từng viewport thiết bị.

## TÀI LIỆU THAM KHẢO

[1] Vercel, ”Next.js 16 Documentation - App Router, Server Components and Data Fetching,”2026. [Online]. Available: https://nextjs.org/docs.

[2] Meta Open Source, ”React 19 Server Components and Actions Architecture Guide,”2026. [Online]. Available: https://react.dev.

[3] The Movie Database, ”TMDB API Reference Documentation (v3 REST API),”2026. [Online]. Available: https://developer.themoviedb.org/reference/intro/getting-started.

[4] Tailwind Labs, ”Tailwind CSS v4 Oxide Engine and High-Performance Utility Styling,”2026. [Online]. Available: https://tailwindcss.com/docs.

[5] Google Chrome Developers, ”Optimizing Web Vitals: LCP, FID/INP, and Cumulative Layout Shift,”2026. [Online]. Available: https://web.dev/vitals/.